

# SISTEMAS OPERATIVOS

SEMANA 04 – SESIÓN 01

Gestión de Procesos: Planificación de  
procesos



Universidad  
Tecnológica  
del Perú

# Logro de aprendizaje

Al finalizar la sesión el estudiante logra:

- Ejecutará correctamente comandos de inicio en Linux.



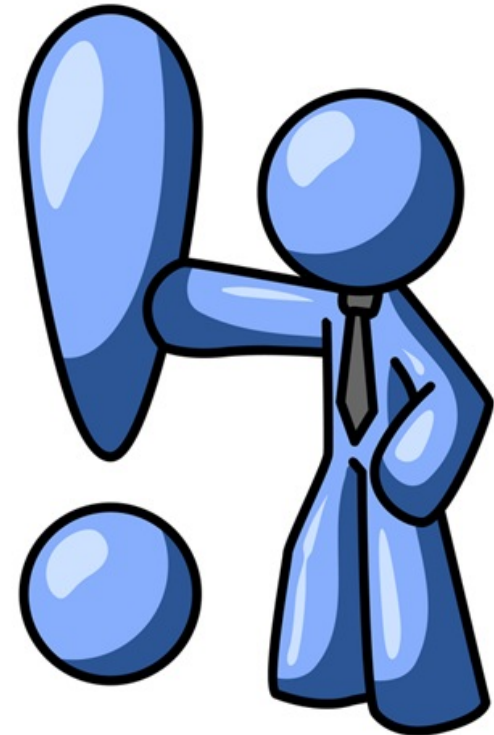
# TEMARIO

- Gestión de Procesos.
- Planificación de procesos.



# Conocimientos previos

- Algoritmos de planificación de procesos.



# UTILIDAD

- Dominar Sistema Operativo.
- Calcular uso de la CPU.
- Entender la prioridad a los procesos.





# ¿Qué es la Gestión de Procesos?

La gestión de procesos es una función fundamental del sistema operativo que se encarga de:

- **Crear y destruir procesos** según las necesidades del sistema
- **Asignar recursos** como CPU, memoria y dispositivos de E/S
- **Coordinar la ejecución** de múltiples procesos simultáneamente
- **Garantizar la eficiencia** y el rendimiento del sistema



# ¿Qué es la Gestión de Procesos?



## Concepto Clave

Un **proceso** es un programa en ejecución que incluye el código del programa, los datos actuales, el contador de programa y el estado de los registros del procesador.



# Estados de un Proceso

## Ciclo de Vida de un Proceso

Nuevo → Listo → Ejecutándose → Terminado

 Bloqueado/Esperando





# Estados de un Proceso

- **Nuevo:** El proceso está siendo creado
- **Listo:** El proceso está esperando ser asignado al procesador
- **Ejecutándose:** Las instrucciones están siendo ejecutadas
- **Bloqueado:** El proceso está esperando que ocurra algún evento
- **Terminado:** El proceso ha finalizado su ejecución



# Planificación de Procesos

## Objetivos de la Planificación:

- **Maximizar la utilización de CPU**
- **Maximizar el throughput** (procesos completados por unidad de tiempo)
- **Minimizar el tiempo de respuesta**
- **Minimizar el tiempo de espera**
- **Minimizar el tiempo de retorno**



# Planificación de Procesos



## Tipos de Planificación

**Planificación a Largo Plazo:** Controla qué procesos son admitidos en el sistema

**Planificación a Corto Plazo:** Selecciona qué proceso ejecutar a continuación

**Planificación a Medio Plazo:** Controla el swapping de procesos



# Algoritmo FCFS (First Come First Served)

## Características:

- El primer proceso en llegar es el primero en ser ejecutado
- Algoritmo no expulsivo (non-preemptive)
- Simple de implementar
- Puede causar el "efecto convoy"



# Algoritmo FCFS (First Come First Served)

**¿Ya es hora de que empiece?**

- Si `current_time < process.arrival_time` :
  - El CPU está libre, así que avanza el reloj hasta que llegue el proceso
  - `current_time = process.arrival_time`

**Ejecutar el proceso:**

- `start_time = current_time` (cuándo empezó)
- `finish_time = current_time + process.burst_time` (cuándo termina)
- `waiting_time = start_time - process.arrival_time` (cuánto esperó)
- `turnaround_time = finish_time - process.arrival_time` (tiempo total)



# Algoritmo FCFS (First Come First Served)

**Actualizar el reloj:**

- `current_time = finish_time` (el CPU queda ocupado hasta que termine)



# Algoritmo FCFS (First Come First Served)

## Ejemplo práctico:

Si tienes estos procesos:

- Proceso A: llega en tiempo 0, duración 5
- Proceso B: llega en tiempo 2, duración 3
- Proceso C: llega en tiempo 4, duración 2

## Ejecución:

1. A ejecuta de 0→5 (no espera)
2. B ejecuta de 5→8 (espera 3 unidades:  $5-2=3$ )
3. C ejecuta de 8→10 (espera 4 unidades:  $8-4=4$ )



# Algoritmo FCFS (First Come First Served)

El que llega primero, se ejecuta primero, sin interrupciones.



# **Algoritmo SJF (Shortest Job First)**

## Características:

- Ejecuta primero el proceso con menor tiempo de ráfaga
- Minimiza el tiempo de espera promedio
- Puede ser expulsivo (SRTF) o no expulsivo
- Problema: predicción del tiempo de ejecución

# Algoritmo SJF (Shortest Job First)

## Paso A: Encontrar procesos disponibles

```
python
```

```
available = [p for p in remaining if p.arrival_time <= current_time]
```

- Solo considera procesos que **ya llegaron** al sistema

# Algoritmo SJF (Shortest Job First)

## Paso B: ¿Hay procesos disponibles?

- **SI NO** hay procesos disponibles:
  - `current_time = min(p.arrival_time for p in remaining)`
  - Avanza el tiempo hasta que llegue el próximo proceso

## Paso C: Seleccionar el más corto

```
python  
  
shortest = min(available, key=lambda x: x.burst_time)
```

- De todos los disponibles, elige el que tenga **menor duración**

# Algoritmo SJF (Shortest Job First)

## Paso D: Ejecutar el proceso

- `start_time = current_time`
- `finish_time = current_time + shortest.burst_time`
- `waiting_time = start_time - shortest.arrival_time`
- `turnaround_time = finish_time - shortest.arrival_time`

## Paso E: Actualizar estado

- `current_time = finish_time` (avanzar reloj)
- `remaining.remove(shortest)` (quitar de pendientes)
- Agregar resultado a `completed`



# Algoritmo SJF (Shortest Job First)

## Ejemplo práctico:

### Procesos:

- A: llega=0, duración=8
- B: llega=1, duración=4
- C: llega=2, duración=2
- D: llega=3, duración=1

### Ejecución:

1. Tiempo 0: Solo A está disponible → ejecuta A (0→8)? **NO**
2. Tiempo 1: A y B disponibles → B es más corto (4<8) → ejecuta B (1→5)
3. Tiempo 5: A, C, D disponibles → D es más corto (1) → ejecuta D (5→6)
4. Tiempo 6: A, C disponibles → C es más corto (2<8) → ejecuta C (6→8)
5. Tiempo 8: Solo A queda → ejecuta A (8→16)

# Algoritmo SJF (Shortest Job First)

**¡La clave es siempre elegir el trabajo más corto entre los que ya llegaron!**



# Algoritmo Round Robin

## Características:

- Cada proceso recibe un quantum de tiempo fijo
- Algoritmo expulsivo y circular
- Excelente para sistemas interactivos
- El rendimiento depende del tamaño del quantum



# Algoritmo Round Robin

## Paso A: Tomar el siguiente proceso

```
python  
process = queue.pop(0) # Saca el primero de la cola
```

## Paso B: ¿Ya llegó este proceso?

```
python  
  
if process.remaining_time <= quantum:  
    # El proceso TERMINA en este turno
```

## Paso C: Verificar si el proceso termina





# Algoritmo Round Robin

## Paso C: Verificar si el proceso termina

**SI TERMINA** ( $\text{remaining\_time} \leq \text{quantum}$ ):

- `current_time += process.remaining_time` (usa solo el tiempo que necesita)
- `turnaround = current_time - process.arrival_time`
- `waiting = turnaround - process.burst_time` (tiempo original)
- Agregar a `completed`



# Algoritmo Round Robin

**SI NO TERMINA** (`remaining_time > quantum`):

- `current_time += quantum` (usa todo el quantum)
- `process.remaining_time -= quantum` (reduce tiempo restante)
- `queue.append(process)` (va al final de la cola)



# Algoritmo Round Robin

## Paso D: Agregar nuevos procesos

```
python
```

```
new_arrivals = [p for p in processes if process.arrival_time <= current_time]  
queue.extend(new_arrivals)
```



# Algoritmo Round Robin

## Ejemplo práctico (quantum = 3):

Procesos:

- A: llega=0, duración=8
- B: llega=1, duración=4
- C: llega=2, duración=2

## Ejecución:

1. **Tiempo 0-3:** A ejecuta 3 unidades (quedan 5) → A va al final
2. **Tiempo 3-6:** B ejecuta 3 unidades (queda 1) → B va al final
3. **Tiempo 6-8:** C ejecuta 2 unidades → **C TERMINA**
4. **Tiempo 8-9:** B ejecuta 1 unidad → **B TERMINA**
5. **Tiempo 9-12:** A ejecuta 3 unidades (quedan 2) → A va al final
6. **Tiempo 12-14:** A ejecuta 2 unidades → **A TERMINA**



# Algoritmo Round Robin

**¡Es como una rueda que gira dando oportunidades iguales a todos!**



# Algoritmo por Prioridades

## Características:

- Cada proceso tiene asignada una prioridad
- Se ejecuta el proceso con mayor prioridad
- Puede ser expulsivo o no expulsivo
- Problema: inanición (starvation)



# Algoritmo por Prioridades

## Paso A: Encontrar procesos disponibles

python

```
available = [p for p in remaining if p.arrival_time <= current_time]
```



# Algoritmo por Prioridades

## Paso B: ¿No hay procesos disponibles?

- Avanza el tiempo hasta que llegue el próximo proceso

## Paso C: Seleccionar mayor prioridad

```
python
```

```
highest_priority = min(available, key=lambda x: x.priority)
```

- **min()** porque menor número = mayor prioridad





# Algoritmo por Prioridades

## Paso D: Ejecutar completamente

- `start_time = current_time`
- `finish_time = current_time + highest_priority.burst_time`
- `waiting_time = start_time - highest_priority.arrival_time`
- El proceso se ejecuta **sin interrupciones** hasta terminar



# Algoritmo por Prioridades

## Ejemplo práctico:

Procesos:

- A: llega=0, duración=8, prioridad=3
- B: llega=1, duración=4, prioridad=1 (¡mayor prioridad!)
- C: llega=2, duración=2, prioridad=2



# Algoritmo por Prioridades

## Ejemplo práctico:

Procesos:

- A: llega=0, duración=8, prioridad=3
- B: llega=1, duración=4, prioridad=1 (¡mayor prioridad!)
- C: llega=2, duración=2, prioridad=2

## Ejecución Non-Preemptive:

1. **Tiempo 0-1:** Solo A disponible → ejecuta A...
2. **Tiempo 1:** Llega B (prioridad 1), pero A ya está ejecutando
3. **Tiempo 0-8:** A termina completamente (no se puede interrumpir)
4. **Tiempo 8-12:** B ejecuta (prioridad 1 > prioridad 2 de C)
5. **Tiempo 12-14:** C ejecuta



# Algoritmo por Prioridades

## Ejecución Preemptive sería:

1. **Tiempo 0-1:** A ejecuta
2. **Tiempo 1:** Llega B (prioridad 1) → **interrumpe a A**
3. **Tiempo 1-5:** B ejecuta y termina
4. **Tiempo 5-7:** C ejecuta (prioridad 2)
5. **Tiempo 7-14:** A reanuda y termina



# Algoritmo por Prioridades

**¡La clave es que los procesos más importantes siempre van primero!**



# Comparación de Algoritmos

| Algoritmo          | Ventajas             | Desventajas        | Uso Recomendado            |
|--------------------|----------------------|--------------------|----------------------------|
| <b>FCFS</b>        | Simple, justo        | Efecto convoy      | Sistemas batch             |
| <b>SJF</b>         | Tiempo espera mínimo | Predicción difícil | Sistemas conocidos         |
| <b>Round Robin</b> | Interactividad       | Overhead de cambio | Sistemas tiempo compartido |
| <b>Prioridades</b> | Control fino         | Posible inanición  | Sistemas críticos          |



# Comparación de Algoritmos



## Factores de Selección

La elección del algoritmo depende del tipo de sistema, la carga de trabajo esperada, y los objetivos de rendimiento específicos del entorno de ejecución.



# Conclusiones

- FCFS – El que llega primero, se ejecuta primero, sin interrupciones.
- SJF – La clave es elegir el trabajo más corto entre los que ya llegaron.
- Round Robin – Es como una rueda que gira dando oportunidades iguales a todos.
- Algoritmo por Prioridades – La clave es que los procesos más importantes siempre van primero.



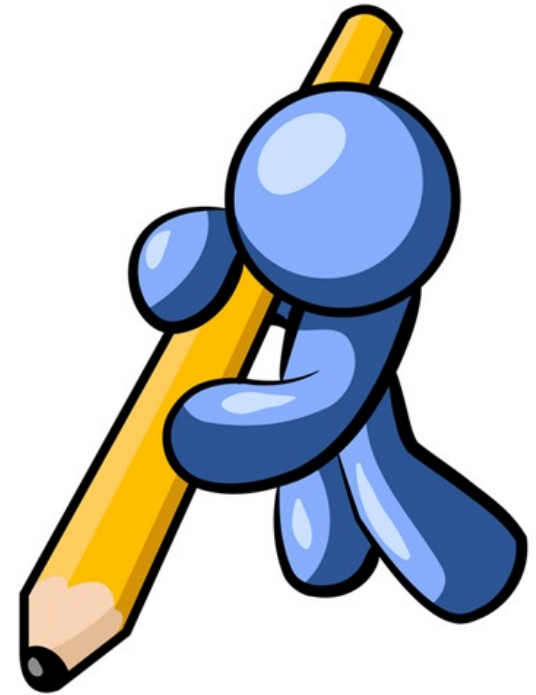
# Preguntas

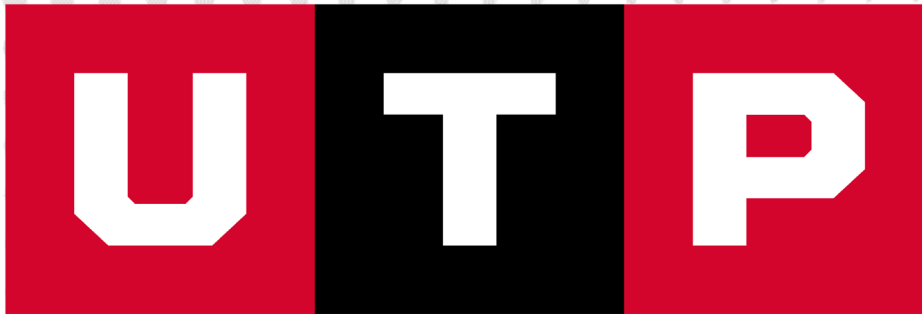
- ¿Tienes alguna duda o comentario?



# CIERRE

- ¿Qué aprendiste en esta sesión?
- Te invitamos a compartir tus conclusiones en clase





**Universidad  
Tecnológica  
del Perú**